

AZURE EVENT GRID EVENT-DRIVEN AUTOMATION

Problem Statement, Objectives and Requirements

1. Project Information

Project Type	Event Architecture
Project Code	24CC3046-P058
Faculty	Challapalli Jhansi Rani
Team Member IDs	2400032490 – Tungala Naga Venkata Siva Sai 2400032479 – Vignathri Kannikanti 2400032462 – Ms. Mudra Swami 2400032413 – Sannidhi Bhavana
Students	4 students

2. Problem Statement

Traditional automation systems often depend on polling-based scripts to repeatedly check Azure resources and storage changes. This approach can introduce unnecessary delays, consume resources, and make automation difficult to maintain. The proposed system uses Azure Event Grid to detect resource and storage events and automatically trigger appropriate actions in real time. The solution should also address duplicate processing caused by event delivery retries and failures caused by changes in event schemas.

3. Objectives

- Implement event-driven automation using Azure Event Grid.
- Trigger automation automatically when relevant Azure resource or storage events occur.
- Reduce or replace polling-based automation scripts.
- Enable near real-time processing events.
- Handle event delivery retries and minimizes duplicate processing.
- Design event handlers that can safely handle event schema changes.
- Improve the scalability, reliability, and maintainability of the automation solution.

4. Functional Requirements

- The system shall receive events from supported Azure resources and storage services.
- The system shall use Azure Event Grid to route events to the appropriate event handler.
- The system shall trigger the required automation when a valid event is received.
- The system shall process events without continuously polling the source resources.
- The system shall identify and safely handle repeated event deliveries.
- The system shall validate incoming event data before processing.
- The system shall log event processing results and failures.
- The system shall support error handling and retry mechanisms for failed event processing.

5. Non-Functional Requirements

- Scalability: The solution should support increasing event volumes.
- Reliability: Events should be processed consistently even when temporary failures occur.
- Fault Tolerance: The system should recover from processing or delivery failures.
- Performance: Events should trigger automation with low processing delay.
- Security: Only authorized services and event sources should be allowed to trigger automation.
- Maintainability: Event handlers should be modular and easy to update.
- Monitoring: Event delivery and processing should be observable through logs and monitoring tools.

6. Use Cases

- Trigger automation when an Azure resource event occurs.
- Trigger automation when a storage event occurs, such as creation or modification of an object.
- Replace polling-based scripts with event-based triggers.
- Route different event types to appropriate processing logic.
- Monitor and handle failed or repeated event deliveries.

7. Potential Bottlenecks / Challenges

- Event delivery retries may cause the same event to be processed more than once.
- Changes in the event schema may cause existing event handlers to fail silently.
- High event volumes may require scalable event-processing components.
- Temporary service or network failures may delay event processing.
- Incorrect event filtering or routing may trigger unwanted automation.

8. Expected Outcome

The project is expected to provide a reliable event-driven automation solution using Azure Event Grid. It should automatically respond to Azure resource and storage events, reduce dependence on polling scripts, support near real-time processing, and provide mechanisms for handling duplicate events, failures, and event schema changes.

9. Suggested Azure Services / Technologies

- Azure Event Grid – event routing and event delivery.
- Azure Functions – event-driven processing and automation logic.
- Azure Storage – source for storage-related events.
- Azure Monitor / Application Insights – monitoring, logging, and troubleshooting.

10. Conclusion

Azure Event Grid provides an event-driven approach for building responsive and scalable automation. By replacing continuous polling with event-based triggers, the proposed system can improve efficiency while maintaining reliable event processing and easier maintenance.